

实 验 报 告

课程名称：机器学习及优化

姓名：王志豪

学号：08232337

班级：2023 大数据一班

日期：2026. 4. 4

中国矿业大学计算机学院/人工智能学院

实验名称 逻辑回归与朴素贝叶斯分类 实验日期 2026. 4. 4

实验报告内容: 1. 实验目的 2. 实验内容 3. 实验步骤 4. 实验结果 5. 实验体会

一、实验目的

通过实验更好地掌握机器学习及优化的相关概念、技术、原理、应用等；通过实验提高学生编写实验报告、总结实验结果的能力；使对机器学习模型、优化算法等有比较深入的认识。

需掌握的知识点：

- (1) 掌握机器学习及优化中涉及的相关概念、模型、算法；
- (2) 熟悉机器学习模型训练、验证、测试的流程；
- (3) 熟悉常用的数据预处理方法；
- (4) 掌握逻辑回归、贝叶斯分类的表示、求解及编程。

二、实验内容

基于 IRIS 鸢尾花数据集，完成样本是否属于维吉尼亚鸢尾(Iris-Virginica)的逻辑回归分类、朴素贝叶斯模型训练、测试与评估。

- (1) 准备数据集并认识数据 下载 IRIS 数据集 <https://archive.ics.uci.edu/ml/datasets/iris> 了解数据集各个维度特征及预测值的含义
- (2) 探索数据并预处理数据 观察数据集各个维度特征及预测值的数值类型与分布
- (3) 训练模型 编程实现训练数据集上逻辑回归模型的梯度下降参数求解、朴素贝叶斯参数统计
- (4) 测试和评估模型 在测试数据集上计算所训练模型的准确率、F1-score 等指标

三、实验步骤

实验原理：

- (1) 逻辑回归分类

A. 模型表示

给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ ，其中 $x_i = (x_{i1}; x_{i2}; \dots; x_{id})$ ， $y_i \in \{0, 1\}$ ，参数 $\theta = (\theta_0; \theta_1; \dots; \theta_d)$ 。

逻辑回归通过 Sigmoid 函数将线性回归的输出映射到 $(0, 1)$ 区间，表示样本属于正类的概率：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

因此，预测函数为：

$$h_{\theta}(x_i) = \frac{1}{1 + e^{-\theta^T x_i}}$$

B. 损失函数

采用对数损失函数（交叉熵损失），评估预测值与真实标签之间的差异：

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))]$$

C. 梯度下降优化

通过梯度下降法最小化损失函数，参数更新公式为：

$$\theta = \theta - \eta \nabla J(\theta)$$

对于每个分量 θ_j ：

$$\theta_j = \theta_j - \eta \cdot \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i) x_{ij}$$

向量化形式为：

$$\theta = \theta + \eta \cdot \frac{1}{N} X^T (y - \sigma(X\theta))$$

D. 梯度推导（链式法则）

令 $z_i = \theta^T x_i$ ，则：

$$\frac{\partial J}{\partial \theta} = \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i) x_i$$

对于偏置项 b ：

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i)$$

逻辑回归是一种用于二分类问题的线性模型。它通过将特征的线性组合输入 **Sigmoid** 函数，将输出值转换为 0 到 1 之间的概率，从而判断样本属于某一类别的可能性。模型通过梯度下降法最小化交叉熵损失函数，不断优化参数，提高分类准确性。逻辑回归具有计算效率高、可解释性强等优点，广泛应用于二分类任务。

（2）朴素贝叶斯分类

A. 贝叶斯定理

对于某个样本 $X = (x_1, x_2, \dots, x_d)$ ，估计它属于类别 C_k 的概率为：

$$P(C_k | X) = \frac{P(X | C_k)P(C_k)}{P(X)}$$

B. 条件独立性假设

朴素贝叶斯假设各特征之间相互独立（条件独立）：

$$P(X | C_k) = \prod_{i=1}^d P(x_i | C_k)$$

C. 高斯朴素贝叶斯

对于连续特征，假设每个特征在给定类别下服从正态分布：

$$P(x_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} e^{-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}}$$

其中:

- $\mu_{k,i}$: 类别 C_k 下特征 x_i 的均值
- $\sigma_{k,i}^2$: 类别 C_k 下特征 x_i 的方差

D. 参数估计（极大似然估计）

$$\mu_{k,i} = \frac{1}{N_k} \sum_{j=1}^{N_k} x_{j,i}$$

其中 N_k 为类别 k 下的样本数。

E. 分类决策

计算每个类别的后验概率，选择最大后验概率对应的类别：

$$\text{Class}(X) = \arg \max_k \left[P(C_k) \prod_{i=1}^d P(x_i | C_k) \right]$$

朴素贝叶斯是一种基于贝叶斯定理的概率分类方法。它假设各特征在给定类别下相互独立（条件独立性假设），从而将联合概率分解为单个特征概率的乘积。对于连续特征，通常采用高斯分布来估计每个特征在各类别下的概率密度。模型通过训练数据计算各类别的先验概率以及每个特征的条件概率，然后利用贝叶斯定理计算后验概率，最终将样本划分到后验概率最大的类别中。朴素贝叶斯方法简单、计算效率高，对小规模数据表现良好。

（3）度量指标

精确率 Precision: 预测为正类的样本中，实际为正类的比例。

$$\text{Precision} = \frac{TP}{TP + FP}$$

召回率 Recall: 实际为正类的样本中，预测为正类的比例。

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score: 精确率和召回率的调和平均数。

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

1. 数据集准备与探索

```
数据列名: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
类别列唯一值: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Sepal_Length: 花萼长度，即花萼的最长部分，单位是厘米。

Sepal_Width: 花萼宽度，即花萼的最宽部分，单位是厘米。

Petal_Length: 花瓣长度，即花瓣的最长部分，单位是厘米。

Petal_Width: 花瓣宽度，即花瓣的最宽部分，单位是厘米。

标签列 Species 中的类别有：

Iris-setosa: 山鸢尾

Iris-virginica: 维吉尼亚鸢尾

Iris-versicolor: 变色鸢尾

2. 预处理

(1) 空值处理

统计空值数量：

```
空值数量如下：
sepal_length      0
sepal_width       0
petal_length      0
petal_width       0
species           0
dtype: int64
```

可见此数据集不存在空值，因此无需处理。

(2) 归一化

先对每一列做统计分析，观察其分布情况以及列与列之间的差异。如果某一列的数值变化很小（例如几乎都是相同的值），说明该特征信息量有限，可以考虑删除。通过分位数还能识别是否存在离群点，从而决定是否进行处理或剔除。另外，如果不同特征之间的取值范围差异较大（如一个在 0-1，另一个在 100-200），则需要归一化或标准化，以避免量纲差异影响模型对特征重要性的判断。

各列数据数值分布为

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

但可以发现，这里各列量纲相近，因此选择不归一化或者标准化。

(3) 处理“品种 species”列

原始数据为：

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

将不是 virginica 的映射为 0，将 virginica 的映射为 1:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	1
146	6.3	2.5	5.0	1.9	1
147	6.5	3.0	5.2	2.0	1
148	6.2	3.4	5.4	2.3	1
149	5.9	3.0	5.1	1.8	1

统计数量为:

```
非 virginica 的数量: 100
virginica 的数量: 50
```

(4) 划分数据集

划分 70%的样本作为训练数据集，30%的样本作为测试数据集

```
原始数据量150
训练集数据量105
测试集数据量45
```

将所有数据整合成 x-y 的形式 (y 就是之前置于尾部的 species)

X 矩阵为	y 矩阵为
[5.5 2.4 3.7 1.]	[0]
[6.3 2.8 5.1 1.5]	[1]
[6.4 3.1 5.5 1.8]	[1]
[6.6 3. 4.4 1.4]	[0]
[7.2 3.6 6.1 2.5]	[1]
[5.7 2.9 4.2 1.3]	[0]
[7.6 3. 6.6 2.1]	[1]
[5.6 3. 4.5 1.5]	[0]
[5.1 3.5 1.4 0.2]	[0]
[7.7 2.8 6.7 2.]	[1]
[5.8 2.7 4.1 1.]	[0]
[5.2 3.4 1.4 0.2]	[0]
[5. 3.5 1.3 0.3]	[0]
[5.1 3.8 1.9 0.4]	[0]
[5. 2. 3.5 1.]	[0]

3. 逻辑回归模型的梯度下降参数求解

首先创建 `theta`，注意不要创建成一维的零数组，正确创建方法如下：

```
theta = np.zeros((X.shape[1], 1)) # 创建一个列向量
```

定义梯度下降函数：

```
# 梯度下降函数
def gradient_descent(theta, alpha):
    for i in range(iter):
        theta = theta - (alpha / m) * (X.T @ (sigmoid(X @ theta) - y))
    return theta
```

将 `theta` 代入梯度下降函数，而后我们打印 `X`，`Y` 以及 `theta` 的形状：

```
y 的形状为(105, 1)
X 的形状为(105, 5)
theta 的形状为(5, 1)
```

而后将 `X_test` 与 `theta` 代入 `sigmoid` 函数后得到 `pred` 的结果，我们分别打印 `pred` 的形状以及具体内容：

```
*pred 的形状为(1, 45)
[[0.41595473 0.00792698 0.88203174 0.41895109 0.36344209 0.01114369
 0.22909819 0.55115323 0.50527238 0.27759524 0.54480488 0.01575982
 0.00664436 0.0153354 0.00962996 0.37439148 0.7793172 0.3196367
 0.44904692 0.77678952 0.01982443 0.55424945 0.01641284 0.76283774
 0.55909008 0.62888416 0.76048037 0.75077471 0.0183988 0.01981798
 0.00830466 0.00452481 0.24081806 0.01613576 0.01751676 0.6496227
 0.30644767 0.0102239 0.00981039 0.00624254 0.71919651 0.35499304
 0.32452807 0.00631318 0.00774845]]
```

使用通过 `numpy` 的广播和布尔索引功能自动进行逐元素比较和计数因为 `numpy` 的布尔数组可以直接用于逐元素比较，它会生成一个布尔值数组，表示每个元素的匹配情况。最后得到的 45 个预测概率如下，将概率高于 0.5 的处理为 1，其余为 0，我们分别打印出预测序列以及真值序列：

```

预测序列为
[[0 0 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1
  0 0 0 0 1 0 0 0 0]]
真值序列为
[[0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1
  0 0 0 0 1 0 0 0 0]]

```

可以从上图中发现有一个错误，而后我们打印出计算的评测指标数值：

```

正确预测个数为 44 个，总个数有 45 个
正确率为 0.9777777777777777
TP 为13
FP 为1
FN 为0
TN 为31
Precision: 0.9285714285714286
Recall: 1.0
F1-score: 0.962962962962963
accuracy: 0.9777777777777777

```

因为在实验结果中， $FP = 1$ ， $TP = 13$ ， $TN = 31$ ， $FN = 0$ ，则有：

$$Precision = \frac{TP}{TP + FP} = \frac{13}{13 + 1} = \frac{13}{14} \approx 0.97778$$

$$Recall = \frac{TP}{TP + FN} = \frac{13}{13 + 0} = 1.0$$

$$F1 - Score = 2 \times \frac{\frac{13}{14} \times 1}{\frac{13}{14} + 1} \approx 0.96296$$

与代码结果一致。

4.朴素贝叶斯参数统计

注意将 y 的形状进行展平：

```

y = y.flatten()
# 计算每个特征的均值和标准差
mean_v = X[:, 1:][y == 1].mean(axis=0)

```

计算每个类别的先验概率以及每个特征的均值和标准差：

```

# 计算每个类别的先验概率
v_prior = np.mean(y) # Virginica 的先验概率
nv_prior = 1 - v_prior # 非 Virginica 的先验概率
y = y.flatten()
# 计算每个特征的均值和标准差
mean_v = X[:, 1:][y == 1].mean(axis=0)
std_v = X[:, 1:][y == 1].std(axis=0)
mean_nv = X[:, 1:][y == 0].mean(axis=0)
std_nv = X[:, 1:][y == 0].std(axis=0)

```

定义高斯分布概率密度函数以及预测函数：


```
#高斯分布概率密度函数
def gaussian_prob(x, mean, std):
    return (1 / (np.sqrt(2 * np.pi) * std)) * np.exp(-0.5 * ((x - mean) / std) ** 2)
# 预测函数
def predict(X):
    predictions = []
    for x in X:
        # 计算每个特征在 Virginica 类别下的条件概率
        prob_v = np.prod(gaussian_prob(x, mean_v, std_v)) * v_prior
        # 计算每个特征在非 Virginica 类别下的条件概率
        prob_nv = np.prod(gaussian_prob(x, mean_nv, std_nv)) * nv_prior
        # 比较两个类别的概率，取概率大的类别
        predictions.append(1 if prob_v > prob_nv else 0)
    return np.array(predictions)
```

预测结果为：

```
原始真值序列为
[0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1 0
 0 0 0 1 0 0 0 0]
预测序列为
[0 0 1 0 1 0 0 1 0 0 1 0 0 0 0 1 1 0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 1 1
 0 0 0 1 0 1 0 0]
```

可以发现一共有 4 处错误，打印出最终的评测指标结果：

```
正确预测个数为 41 个，总个数有 45 个
正确率为 0.9111111111111111
TP 为13
FP 为4
FN 为0
TN 为28
Precision: 0.7647058823529411
Recall: 1.0
F1-score: 0.8666666666666666
accuracy: 0.9111111111111111
```

因为在实验结果中， $FP = 4$ ， $TP = 13$ ， $TN = 28$ ， $FN = 0$ ，则有：

$$Precision = \frac{TP}{TP + FP} = \frac{13}{13 + 4} = \frac{13}{17} \approx 0.76471$$

$$Recall = \frac{TP}{TP + FN} = \frac{13}{13 + 0} = 1.0$$

$$F1 - Score = 2 \times \frac{\frac{13}{17} \times 1}{\frac{13}{17} + 1} \approx 0.86667$$

与代码输出一致。

5.展示与讨论实验结果

指标	梯度下降法	朴素贝叶斯
正确预测个数	44	41
总个数	45	45

正确率	0.9777777777777777	0.9111111111111111
TP	13	13
FP	1	4
FN	0	0
TN	31	28
Precision	0.9285714285714286	0.7647058823529411
Recall	1.0	1.0
F1-score	0.962962962962963	0.8666666666666666
Accuracy	0.9777777777777777	0.9111111111111111

从两种方法的实验结果来看，**梯度下降法**与**朴素贝叶斯**在识别 Iris-Virginica 这一类别上均表现出色，尤其在召回率（Recall）方面，两者均达到了 **1.0**，说明它们都能够完整地识别出所有正样本，没有任何漏报（FN=0）。

具体来看：

- **梯度下降法**的整体表现更为优异。其正确预测个数为 **44**，总样本数为 **45**，整体准确率高达 **97.8%**，仅有一个样本预测错误。在正样本预测方面，精确率（Precision）达到 **92.9%**，F1 分数为 **96.3%**。这表明梯度下降法在识别 Iris-Virginica 时既稳定又精准，误报极少，堪称“稳准狠”。
- **朴素贝叶斯**同样表现良好，正确预测个数为 **41**，整体准确率为 **91.1%**。在正样本识别上，精确率为 **76.5%**，F1 分数为 **86.7%**。尽管其精确率低于梯度下降法，但召回率同样为满分，说明该模型对正类样本的识别能力也非常可靠，整体性能依然优秀。

综合结论：

两种方法在正样本的召回率上表现一致，均为 **100%**，说明它们都能有效捕捉到目标类别。但从整体准确率、精确率和 F1 分数来看，**梯度下降法在本任务中略占优势**，具有更高的预测精度和更低的误报率。若在实际应用中更看重正样本预测的可信度（即减少误报），梯度下降法是更优选择；若更看重模型的简单性与可解释性，朴素贝叶斯仍然是一个非常有竞争力的基线模型。

四、实验结果

梯度下降法：

```
正确预测个数为 44 个，总个数有 45 个
正确率为 0.9777777777777777
TP 为13
FP 为1
FN 为0
TN 为31
Precision: 0.9285714285714286
Recall: 1.0
F1-score: 0.962962962962963
accuracy: 0.9777777777777777
```

朴素贝叶斯法：

```
正确预测个数为 41 个，总个数有 45 个
正确率为 0.9111111111111111
TP 为13
FP 为4
FN 为0
TN 为28
Precision: 0.7647058823529411
Recall: 1.0
F1-score: 0.8666666666666666
accuracy: 0.9111111111111111
```

两种方法在正样本召回率上均达到 100%，识别能力相当。但从整体准确率、精确率和 F1 分数来看，梯度下降法（准确率 97.8%，精确率 92.9%，F1 96.3%）明显优于朴素贝叶斯（准确率 91.1%，精确率 76.5%，F1 86.7%）。因此，在本任务中梯度下降法表现更优，预测更“稳准狠”。（具体分析可见实验步骤 5 的最终分析）

五、实验体会

通过本次实验，我深入掌握了逻辑回归（梯度下降法）与朴素贝叶斯两种分类模型在鸢尾花数据集上的完整应用流程，包括数据探索、预处理、模型训练、参数求解以及性能评估。实验中我认识到，即使数据集较为规整、无空值且量纲相近，仍需仔细处理标签映射并合理划分训练集与测试集。在实现逻辑回归时，理解梯度下降的迭代原理以及 Sigmoid 函数的概率意义至关重要，而朴素贝叶斯则让我体会到基于高斯分布假设的概率统计方法在分类问题中的简洁与高效。从实验结果来看，两种方法的召回率均达到 100%，说明它们对正类样本（Iris-Virginica）的识别能力都很强；但梯度下降法在准确率（97.8%）、精确率（92.9%）和 F1 分数（96.3%）上明显优于朴素贝叶斯（准确率 91.1%、精确率 76.5%、F1 86.7%），这表明逻辑回归在本任务中对误报的控制更好，预测更“稳准狠”。通过对比分析，我进一步理解了不同模型在相同数据上的表现差异及其原因，也体会到模型选择需结合实际应用中对精确率与召回率的侧重。此外，本次实验也锻炼了我编写代码、调试参数以及规范撰写实验报告的能力，为今后更复杂的机器学习任务打下了坚实基础。

教师评语：

指导教师：

日 期：